



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +



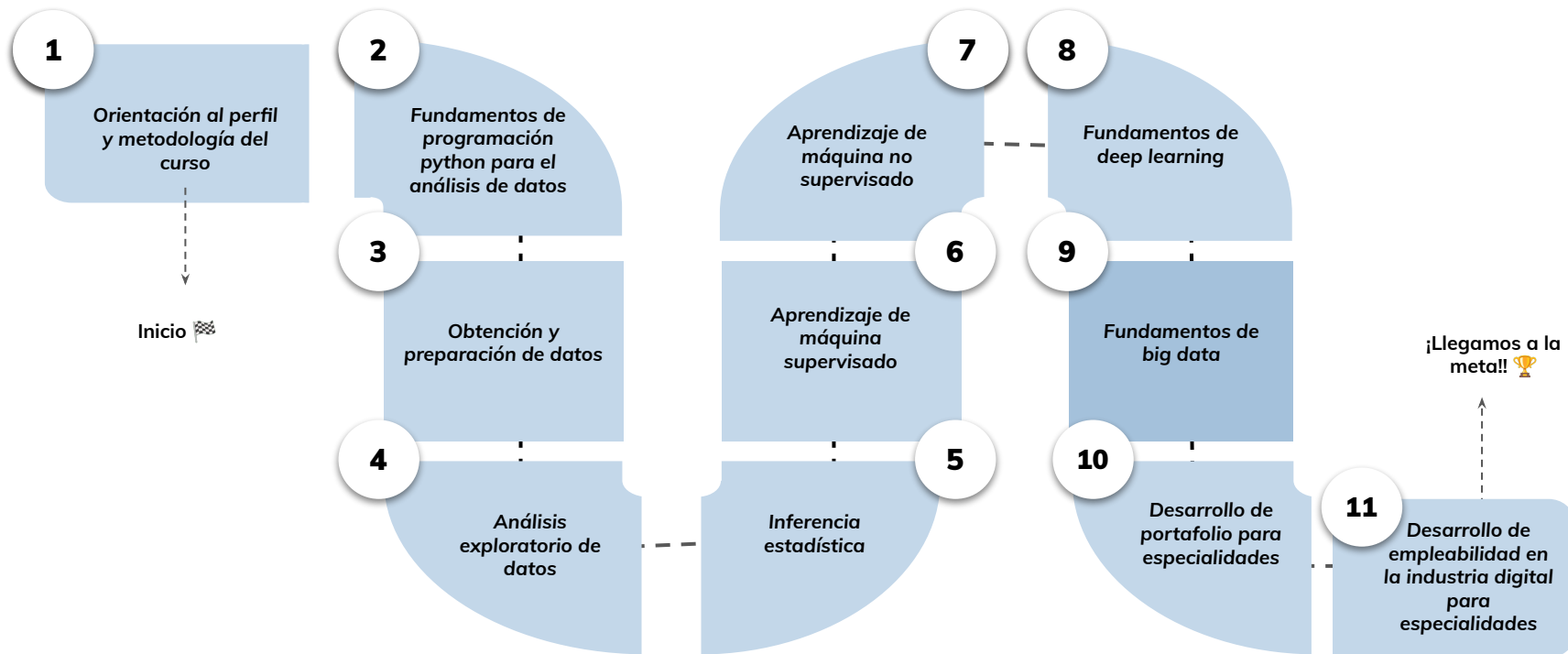
Elementos básicos de spark-

› Parte II

Aprendizaje Esperado 3: Manipular grandes volúmenes de datos utilizando objetos rdd, transformaciones y acciones para resolver un problema en el entorno spark.

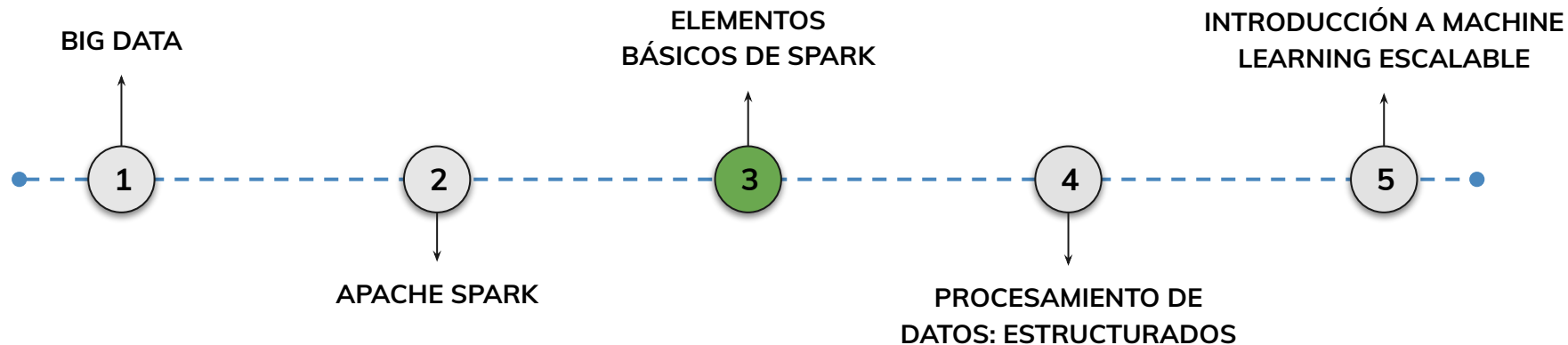
Hoja de ruta

¿Cuáles skills conforman el programa? Fundamentos de Ciencia de Datos



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

3.

Optimización y funcionalidades avanzadas con RDDs en Spark

Profundizaremos en técnicas para optimizar el uso de RDDs, trabajar con datos en distintos formatos y aprovechar funcionalidades avanzadas de Spark como variables broadcast, acumuladores y particionamiento personalizado.

Optimización y buenas prácticas en RDDs

Diferencia entre groupByKey y reduceByKey.

Minimizar shuffle y mejorar rendimiento.

Funcionalidades y extensiones de Spark

Variables broadcast y acumuladores.

Lectura y escritura en múltiples formatos (CSV, JSON, Parquet).

Objetivos de aprendizaje

¿Qué aprenderás?

- Optimizar operaciones en RDD minimizando shuffle.
- Utilizar variables broadcast y acumuladores para mejorar el rendimiento.
- Leer y escribir datos en distintos formatos desde Spark.
- Integrar RDD con Spark SQL y otros módulos.
- Aplicar buenas prácticas de monitoreo, depuración y rendimiento.

Repaso clase anterior

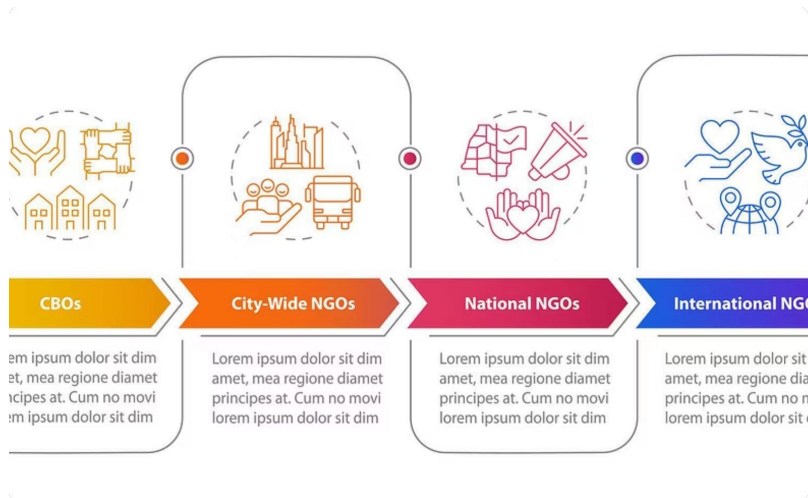
¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprendimos qué es un RDD y su función en Spark.
- Aprendimos a crear RDDs desde colecciones y archivos externos.
- Aplicamos transformaciones como `map()`, `filter()` y `reduceByKey()`.
- Ejecutamos acciones como `collect()` y `count()`.

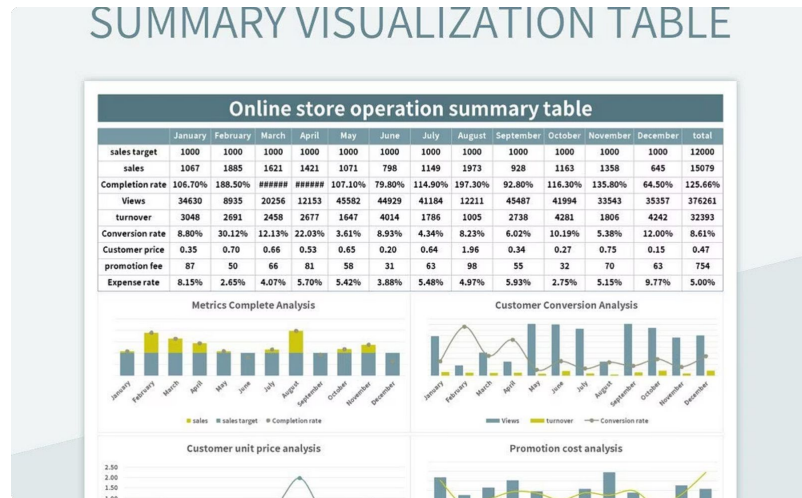
Elementos básicos de spark

Diferencia entre groupByKey y reduceByKey



reduceByKey

Combina datos localmente antes de transferirlos, lo que reduce significativamente el tráfico de red.



groupByKey

Transfiere todos los valores a través de la red y luego los agrupa, lo que puede causar problemas de rendimiento con grandes conjuntos de datos.

Optimización de Shuffle en Spark

El **shuffle** es la redistribución de datos entre particiones que ocurre durante operaciones como `groupByKey`, `reduceByKey` o `join`.

Impacto en el rendimiento

El shuffle es costoso porque implica E/S de disco, serialización y transferencia de red.

Estrategias de optimización

Usar `reduceByKey` en lugar de `groupByKey` cuando sea posible.

Particionamiento

Utilizar `partitionBy` para controlar la distribución de datos y minimizar shuffles.

Broadcast Variables

Las **variables broadcast** permiten mantener una copia de solo lectura de una variable en cada nodo del clúster, en lugar de enviarla con cada tarea.

```
# Crear variable broadcast broadcastVar =  
sc.broadcast([1, 2, 3])# Acceder al  
valor broadcastVar.value
```



Son útiles para compartir grandes conjuntos de datos de referencia que no cambian durante la ejecución.

Acumuladores

Los **acumuladores** son variables que se pueden "acumular", es decir, agregar mediante una operación asociativa.

```
# Crear acumulador
accum = sc.accumulator(0)
# Función que incrementa el acumulador
def f(x):
    global accum
    accum += x
    return x
# Aplicar función a cada elemento
dd.foreach(f)
# Valor final del acumulador
print(accum.value)
```

Son útiles para contadores y sumas en operaciones distribuidas.

Particionamiento Personalizado

Spark permite definir cómo se distribuyen los datos entre las particiones mediante el particionamiento personalizado.

```
# Crear un RDD con particionamiento personalizado  
rdd = pairs.partitionBy(4, lambda key: hash(key) % 4)
```

Un buen particionamiento puede reducir significativamente el shuffle y mejorar el rendimiento.

Lectura de Datos desde Archivos



Archivos de texto

```
lines = sc.textFile("datos.txt")
```



JSON

```
from pyspark.sql import SparkSession  
spark = SparkSession.builder.getOrCreate()  
df = spark.read.json("datos.json")
```



CSV

```
df = spark.read.csv("datos.csv", header=True)
```



Parquet

```
df = spark.read.parquet("datos.parquet")
```

Escritura de Datos a Archivos



Texto

```
rdd.saveAsTextFile("salida")
```



SequenceFile

```
rdd.saveAsSequenceFile("salida")
```



Hadoop

```
rdd.saveAsHadoopFile("salida")
```



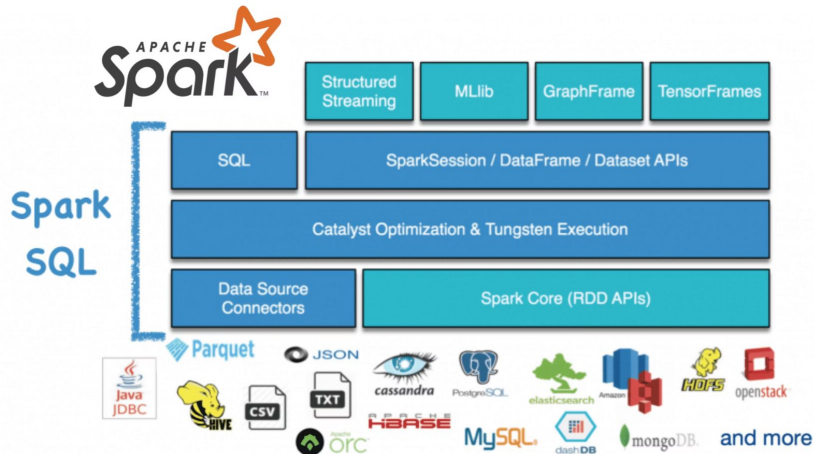
DataFrames

```
df.write.parquet("salida.parquet")
```

Spark SQL: Integración con RDDs

Spark SQL permite trabajar con datos estructurados utilizando SQL dentro de aplicaciones Spark.

```
# Crear DataFrame desde RDD
from pyspark.sql import Row
RowPerson = Row("name", "age")
people = rdd.map(lambda r: RowPerson(r[0], r[1]))
df = spark.createDataFrame(people)
# Registrar como tabla temporal
df.createOrReplaceTempView("people")
# Ejecutar consulta SQL
result = spark.sql("SELECT * FROM people WHERE age > 20")
```



Spark Streaming: Procesamiento en Tiempo Real

Spark Streaming permite procesar datos en tiempo real desde diversas fuentes como Kafka, Flume o TCP sockets.

```
from pyspark.streaming import StreamingContext# Crear contexto de streaming con intervalo de 1
segundosscc = StreamingContext(sc, 1)# Crear DStream desde socketlines =
ssc.socketTextStream("localhost", 9999)# Contar palabraswords = lines.flatMap(lambda line: line.split("
"))pairs = words.map(lambda word: (word, 1))wordCounts = pairs.reduceByKey(lambda a, b: a + b)#
Imprimir resultadoswordCounts.pprint()# Iniciar procesamientossc.start()ssc.awaitTermination()
```

MLlib: Biblioteca de Machine Learning

Algoritmos disponibles

- Clasificación: SVM, regresión logística, Naive Bayes
- Regresión: regresión lineal, árboles de decisión
- Clustering: K-means, LDA
- Recomendación: filtrado colaborativo

Ejemplo básico

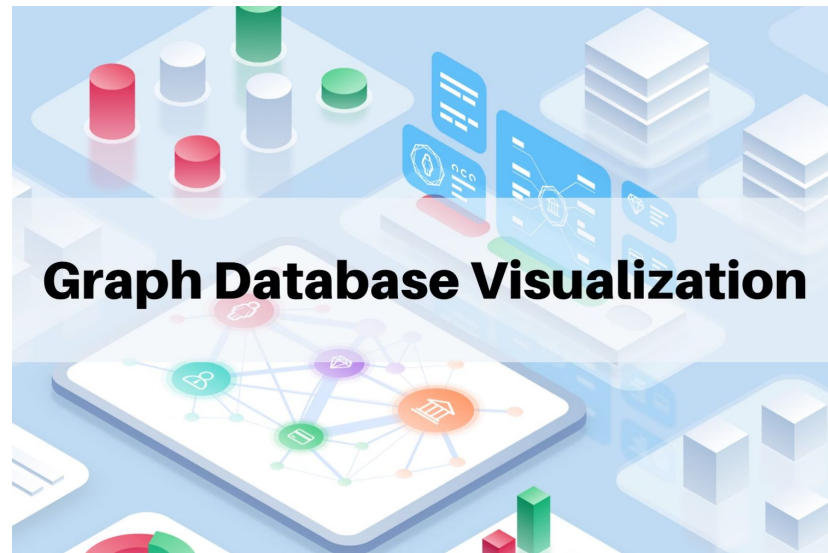
```
from pyspark.mllib.classification import
LogisticRegressionWithSGDfrom
pyspark.mllib.regression import LabeledPoint#
Crear datos de entrenamientodata =
[LabeledPoint(0.0, [0.0, 1.0]),
LabeledPoint(1.0, [1.0, 0.0])]rdd =
sc.parallelize(data)# Entrenar modelomodel =
LogisticRegressionWithSGD.train(rdd)#
Predecirprint(model.predict([1.0, 0.0]))
```

GraphX: Procesamiento de Grafos

GraphX es el componente de Spark para el procesamiento de grafos y la computación paralela de grafos.

Permite construir y analizar grafos dirigidos con propiedades en los vértices y aristas.

- Algoritmos de PageRank
- Componentes conectados
- Caminos más cortos
- Detección de comunidades



Graph Database Visualization

Monitoreo y Depuración



Interfaz Web

Spark proporciona una interfaz web en `http://localhost:4040` para monitorear aplicaciones en ejecución.



Logs

Los logs de Spark contienen información detallada sobre la ejecución de jobs y errores.



Métricas

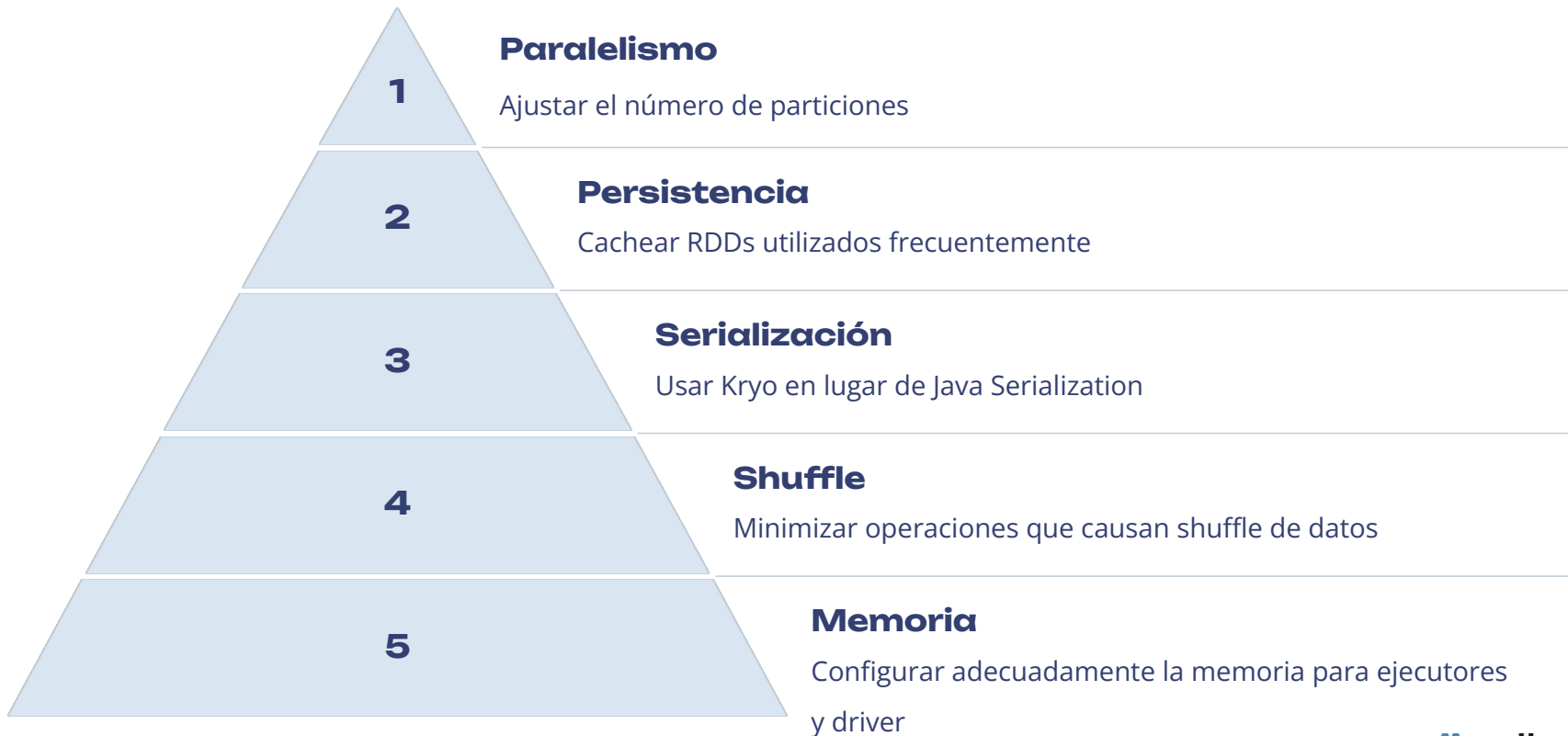
Spark expone métricas a través de JMX que pueden ser recopiladas por sistemas de monitoreo.



Depuración

Utilizar `take()` o `collect()` con pequeñas muestras para verificar resultados intermedios.

Optimización de Rendimiento



Caso Práctico: Análisis de Logs

```
# Cargar logs desde archivo
logs = sc.textFile("logs.txt")
# Filtrar errores
errors = logs.filter(lambda line: "ERROR" in line)
# Extraer códigos de error
errorCodes = errors.map(lambda line: line.split(" ")[3])
# Contar ocurrencias por código
errorCounts = errorCodes.map(lambda code: (code, 1)).reduceByKey(lambda a, b: a + b)
# Ordenar por frecuencia
sortedErrors = errorCounts.sortBy(lambda x: -x[1])
# Mostrar los 10 errores más comunes
print(sortedErrors.take(10))
```

Este ejemplo muestra cómo analizar logs para encontrar los errores más frecuentes.

Caso Práctico: Análisis de Texto

```
# Cargar textotext = sc.textFile("libro.txt")# Dividir en palabras y convertir a minúsculaswords =  
text.flatMap(lambda line: line.lower().split(" "))# Eliminar palabras vacías y signos de  
puntuaciónwords = words.filter(lambda word: len(word) > 0 and word.isalpha())# Contar frecuencia de  
palabraswordCounts = words.map(lambda word: (word, 1)).reduceByKey(lambda a, b: a + b)# Ordenar por  
frecuenciasortedCounts = wordCounts.sortBy(lambda x: -x[1])# Mostrar las 20 palabras más  
comunesprint(sortedCounts.take(20))
```

Caso Práctico: Análisis de Datos CSV

```
# Cargar datos CSV
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()

# Leer CSV
df = spark.read.csv("ventas.csv", header=True, inferSchema=True)

# Registrar como tabla temporal
df.createOrReplaceTempView("ventas")

# Analizar ventas por región
result = spark.sql("""
SELECT region, SUM(monto) as total_ventas
FROM ventas
GROUP BY region
ORDER BY total_ventas
DESC""")

# Mostrar resultados
result.show()
```

Este ejemplo muestra cómo analizar datos de ventas utilizando Spark SQL.

Integración con Hadoop

HDFS

Spark puede leer y escribir datos directamente desde HDFS.

```
rdd =  
sc.textFile("hdfs://namenode:  
8020/data/file.txt")
```

YARN

Spark puede ejecutarse en clústeres YARN como gestor de recursos.

```
spark-submit --master yarn  
--deploy-mode cluster app.py
```

Hive

Spark SQL puede leer y escribir tablas de Hive.

```
from pyspark.sql import  
HiveContext  
hive_context =  
HiveContext(sc)  
result =  
hive_context.sql("SELECT *  
FROM tabla")
```

Despliegue de Aplicaciones Spark

Desarrollo local

Usar modo local para desarrollo y pruebas iniciales.

```
spark-submit --master local[*] app.py
```

1

2

Pruebas en clúster

Probar en un clúster pequeño antes de producción.

```
spark-submit --master spark://master:7077  
app.py
```

3

Producción

Desplegar en clúster de producción con configuración optimizada.

```
spark-submit --master yarn --deploy-mode  
cluster --executor-memory 4g --num-executors  
10 app.py
```

4

Monitoreo

Configurar alertas y monitoreo continuo.

Configuración Avanzada

Memoria

Ajustar `spark.executor.memory` y `spark.driver.memory` según las necesidades de la aplicación.

CPU

Configurar `spark.executor.cores` para controlar el número de tareas por ejecutor.

Shuffle

Optimizar `spark.shuffle.file.buffer` y `spark.reducer.maxSizeInFlight` para mejorar el rendimiento del shuffle.

Serialización

Habilitar Kryo con `spark.serializer=org.apache.spark.serializer.KryoSerializer` para mejorar la eficiencia.

Seguridad en Spark

Autenticación

Configurar autenticación mediante Kerberos o SSL.



Cifrado

Habilitar cifrado en tránsito con SSL/TLS.



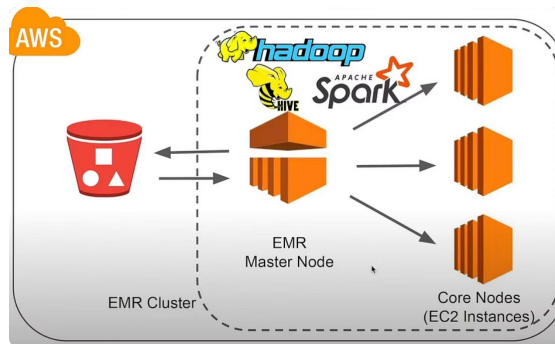
Autorización

Implementar control de acceso basado en roles.

Auditoría

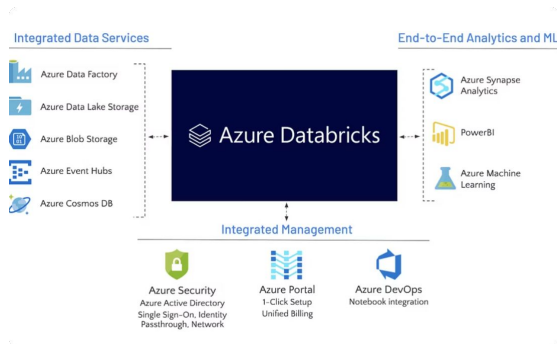
Configurar registro de eventos para auditoría de seguridad.

Spark en la Nube



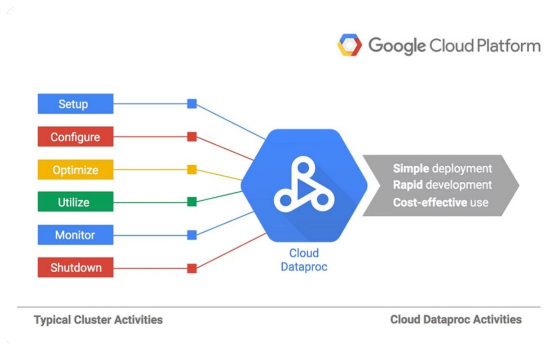
AWS EMR

Amazon Elastic MapReduce ofrece Spark como servicio gestionado con integración con S3, DynamoDB y otros servicios de AWS.



Azure Databricks

Plataforma optimizada para Spark en Azure con características avanzadas de colaboración y seguridad.



Google Dataproc

Servicio gestionado de Spark en Google Cloud con integración con BigQuery, Cloud Storage y otros servicios de GCP.

Comparación con Otras Tecnologías

Spark vs Hadoop MapReduce

Spark es hasta 100 veces más rápido en memoria y tiene una API más rica y fácil de usar.

Spark vs Flink

Flink está más orientado al procesamiento de streams verdaderos, mientras que Spark utiliza micro-batches.

Spark vs Dask

Dask está más integrado con el ecosistema Python (NumPy, Pandas), mientras que Spark tiene un ecosistema más amplio para Big Data.

Tendencias y Futuro de Spark



Computación en la nube

Mayor integración con servicios en la nube y modelos serverless.



IA y ML

Mejoras en las capacidades de machine learning y deep learning.



Streaming

Procesamiento de streams más rápido y con menor latencia.

Kubernetes

Mejor integración con Kubernetes para orquestación de contenedores.

Recursos para Aprender Más



Libros

- Learning Spark (O'Reilly)
- Spark: The Definitive Guide
- Advanced Analytics with Spark



Cursos en línea

- Databricks Academy
- Coursera: Big Data Analysis with Spark
- edX: Distributed Computing with Spark



Comunidad

- Stack Overflow
- Foros oficiales de Apache Spark
- Meetups locales de Spark

Certificaciones de Spark

Databricks Certified Developer

Certifica habilidades fundamentales en desarrollo con Apache Spark.

Databricks Certified Associate

Nivel introductorio para profesionales que trabajan con Spark.

Cloudera CCA Spark and Hadoop Developer

Certifica habilidades en desarrollo con Spark en el ecosistema Hadoop.

Mejores Prácticas

1 **Particionar adecuadamente**

Ajustar el número de particiones según el tamaño de los datos y recursos disponibles.

2 **Persistir RDDs estratégicamente**

Cachear solo los RDDs que se reutilizarán múltiples veces.

3 **Minimizar shuffles**

Reducir operaciones que causan redistribución de datos entre nodos.

4 **Monitorear y ajustar**

Utilizar la interfaz web de Spark para identificar cuellos de botella y optimizar.

5 **Usar formatos de archivo eficientes**

Preferir formatos columnar como Parquet para mejor rendimiento analítico.

Errores Comunes y Soluciones

OutOfMemoryError

Aumentar memoria del driver/ejecutores o reducir paralelismo.

Serialización

Asegurar que todas las clases utilizadas sean serializables.

Skew de datos

Utilizar técnicas como salting para distribuir mejor los datos desbalanceados.

Spill a disco

Optimizar configuración de memoria y shuffle para reducir escrituras a disco.

Comunidad y Soporte

Comunidad de código abierto

Apache Spark tiene una comunidad activa de desarrolladores y usuarios que contribuyen al proyecto y proporcionan soporte a través de:

- Listas de correo
- Foros de discusión
- Repositorio de GitHub
- Conferencias como Spark Summit

Soporte comercial

Varias empresas ofrecen soporte comercial para Spark:

- Databricks (fundada por los creadores de Spark)
- Cloudera
- Hortonworks (ahora parte de Cloudera)
- MapR (ahora parte de HPE)



Conclusión

El manejo de **RDDs en Spark** constituye la base del procesamiento de datos distribuido.

Comprender cómo crear, almacenar y transformar RDDs, junto con la diferencia entre **transformaciones** y **acciones**, permite construir soluciones escalables y eficientes.

El concepto de **lazy evaluation** y **linaje** ofrece tolerancia a fallos y optimizaciones automáticas, mientras que entender cómo se ejecuta un **Job Spark** brinda claridad sobre el flujo de trabajo interno del motor.

Dominar estos elementos es fundamental para desarrollar aplicaciones que manipulen grandes volúmenes de datos de forma eficiente, cumpliendo los objetivos planteados en entornos de **Big Data**.

Live Coding

¿En qué consistirá la Demo?

Analizaremos de forma práctica cómo optimizar operaciones con RDD, aplicar variables broadcast y acumuladores, y leer/escribir datos en distintos formatos dentro de Spark.

1. Comparación práctica de rendimiento entre `groupByKey` y `reduceByKey`.
2. Uso de variables broadcast para compartir datos en el clúster.
3. Implementación de acumuladores para contadores globales.
4. Lectura de datos desde CSV, JSON y Parquet.
5. Escritura de resultados optimizados en formato columnar.



Momento:

Time-out!



5 -10 min.





Ejercicio

Optimización y extensión de RDDs

Optimización y extensión de RDDs

Contexto: 🙌

Vas a aplicar técnicas de optimización y funcionalidades avanzadas de Spark para mejorar el rendimiento del procesamiento distribuido.

Consigna: 📝

1. Crea un RDD y compara el rendimiento de groupByKey y reduceByKey con un dataset mediano.
2. Implementa una variable broadcast para compartir un conjunto de datos de referencia.
3. Usa un acumulador para contar un evento específico durante el procesamiento.
4. Lee datos en formato CSV y escribe resultados en Parquet.

Paso a paso: ⚙️

1. Preparar datos de prueba y configurar Spark.
2. Implementar las operaciones comparativas y medir tiempos.
3. Incorporar broadcast y acumuladores.
4. Guardar los resultados en el formato requerido.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ **Aprendimos a optimizar operaciones en RDD utilizando `reduceByKey` en lugar de `groupByKey`.**
- ✓ **Utilizamos variables broadcast y acumuladores para mejorar el rendimiento.**
- ✓ **Leímos y escribimos datos en formatos CSV, JSON y Parquet.**
- ✓ **Conectamos RDD con Spark SQL y módulos adicionales.**
- ✓ **Aplicamos buenas prácticas de monitoreo y optimización en Spark**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

